Applied AI & Machine Learning Curriculum

From Classical Machine Learning to Deep Learning, Generative AI, Retrieval-Augmented Generation, and Agentic AI Systems

---

A structured, end-to-end learning path covering Machine Learning, Deep Learning, Generative AI, LangChain, LangGraph, LangSmith, Retrieval-Augmented Generation (RAG), Agentic AI design patterns, and Model Evaluation methodology.

Curriculum Document | Version 1.0
Prepared for Self-Paced and Instructor-Led Study

Table of Contents

1. Program Overview and Learning Path

This curriculum provides a complete, sequenced learning path for engineers, data scientists, and product builders who want to move from the fundamentals of machine learning through modern generative AI and into the construction of production-grade agentic systems. It is organized into eleven technical modules plus supporting material on study scheduling, assessment, and reference reading. Each module lists learning objectives, core topics, hands-on exercises, and recommended tools.

The curriculum follows a deliberate progression. It begins with classical machine learning so that learners internalize the statistical reasoning that underlies every later technique, including the neural architectures used in deep learning and the transformer models that power modern generative AI. From there it moves into large language models, prompt design, and retrieval-augmented generation before introducing the orchestration frameworks — LangChain and LangGraph — that are used to assemble multi-step, tool-using applications. The program closes with agentic system design, observability and evaluation using LangSmith, rigorous model evaluation methodology, and the operational practices needed to run these systems reliably in production.

Who this curriculum is for

- Software engineers transitioning into applied machine learning or AI product roles.
- Data scientists who want structured exposure to deep learning and generative AI.
- ML engineers who need practical, tool-level fluency with LangChain, LangGraph, and LangSmith.
- Technical leads evaluating how to design, test, and operate agentic AI systems responsibly.

How to use this document

Work through the modules in order. Each module builds vocabulary and intuition that the next module assumes. Learners with strong prior experience in classical machine learning or deep learning may skip directly to Module C, but should still skim earlier modules for terminology used later in the document. Every module ends with a short set of practice exercises; treat these as mandatory checkpoints rather than optional extras, since the later modules on agentic systems and evaluation assume comfort with the mechanics covered earlier.

2. Foundations: Mathematics and Programming Prerequisites

Before starting the machine learning module, learners should be comfortable with the following foundational areas. These are not treated as full modules because they are widely taught elsewhere, but competence here materially affects how quickly the rest of the curriculum can be absorbed.

Mathematics

- Linear algebra: vectors, matrices, matrix multiplication, eigenvalues and eigenvectors, singular value decomposition, and their role in representing data and model parameters.

- Calculus: derivatives, partial derivatives, the chain rule, and gradients as they apply to optimization.
- Probability and statistics: random variables, distributions, expectation and variance, Bayes' theorem, maximum likelihood estimation, and hypothesis testing.
- Optimization: convexity, gradient descent and its variants, and the intuition behind loss surfaces and local minima.

Programming and Tooling

- Python fluency, including data structures, functions, classes, and package management.
- NumPy and pandas for numerical computing and tabular data manipulation.
- Basic command-line usage, virtual environments, and version control with Git.
- Familiarity with Jupyter notebooks for iterative experimentation.
- Comfort reading API documentation, since later modules rely heavily on third-party libraries such as PyTorch, Hugging Face Transformers, LangChain, and LangGraph.

Learners who need to strengthen these areas should budget one to two weeks before beginning Module A. Trying to learn gradient-based optimization for the first time while simultaneously learning neural network architecture tends to slow progress rather than accelerate it.

3. Module A — Machine Learning Fundamentals

Learning objectives

- Distinguish supervised, unsupervised, and reinforcement learning, and identify which applies to a given problem.
- Build, train, and evaluate classical models for regression and classification tasks.
- Diagnose overfitting and underfitting, and apply appropriate regularization strategies.
- Design a defensible train/validation/test split and cross-validation strategy.

Core topics

3.1 Supervised learning

- Linear and logistic regression, including the derivation of the loss function and gradient-based fitting.
- Decision trees, random forests, and gradient-boosted trees (XGBoost, LightGBM, CatBoost).
- Support vector machines and the kernel trick.
- k-nearest neighbors and distance-based classification.
- Bias-variance tradeoff and the diagnosis of overfitting versus underfitting.

3.2 Unsupervised learning

- Clustering with k-means, hierarchical clustering, and DBSCAN.
- Dimensionality reduction with principal component analysis (PCA) and t-SNE/UMAP for visualization.
- Anomaly detection techniques and their use in fraud and quality-control settings.

3.3 Model evaluation basics

- Accuracy, precision, recall, F1 score, and the ROC/AUC curve for classification.
- Mean squared error, mean absolute error, and R-squared for regression.
- Cross-validation strategies, including k-fold and stratified k-fold.
- Confusion matrices and error analysis by subgroup.

3.4 Feature engineering and data preparation

- Handling missing data, categorical encoding, and feature scaling.
- Feature selection methods and the risks of data leakage.
- Building reproducible preprocessing pipelines.

Hands-on exercises

- Train a logistic regression and a gradient-boosted tree on the same tabular dataset; compare metrics and discuss why the results differ.
- Implement k-fold cross-validation from first principles, without a library helper, to confirm understanding of the mechanics.
- Apply PCA to a high-dimensional dataset and visualize the first two components.

Recommended tools

scikit-learn for classical modeling and evaluation utilities; pandas and NumPy for data handling; matplotlib or seaborn for exploratory visualization.

4. Module B — Deep Learning

Learning objectives

- Explain how a feedforward neural network computes an output and how backpropagation updates its weights.
- Select an appropriate architecture (convolutional, recurrent, or transformer-based) for a given data modality.
- Train a deep model with awareness of common failure modes: vanishing gradients, exploding gradients, and overfitting on limited data.

Core topics

4.1 Neural network fundamentals

- Perceptrons, activation functions (ReLU, sigmoid, tanh, GELU), and the universal approximation intuition.
- Forward propagation and backpropagation, and how automatic differentiation implements the chain rule in frameworks such as PyTorch.

- Loss functions for classification and regression, and how they interact with output layer design.
- Optimizers: stochastic gradient descent, momentum, RMSProp, and Adam.
- Regularization: dropout, weight decay, batch normalization, and early stopping.

4.2 Convolutional neural networks

- Convolution and pooling operations, and why they encode translation invariance.
- Classic architectures (LeNet, VGG, ResNet) and the residual connection as a solution to vanishing gradients in deep networks.
- Transfer learning: fine-tuning a pretrained vision backbone on a new dataset.

4.3 Sequence models

- Recurrent neural networks and long short-term memory (LSTM) networks for sequential data.
- The limitations of recurrence — sequential computation and long-range dependency decay — that motivated the transformer architecture.

4.4 The transformer architecture

- Self-attention and multi-head attention as a mechanism for relating tokens regardless of distance in a sequence.
- Positional encoding, layer normalization, and the encoder-decoder versus decoder-only design choice.
- Why the transformer's parallelizable computation made large-scale pretraining practical, leading directly to today's large language models.

4.5 Training at scale

- Mixed-precision training and gradient checkpointing for memory efficiency.
- Data and model parallelism concepts for distributed training.
- Learning rate schedules, warmup, and the practical effect of batch size on convergence.

Hands-on exercises

- Implement a small feedforward network and its backpropagation update rule from scratch using only NumPy, then confirm the result matches a PyTorch implementation.
- Fine-tune a pretrained convolutional network on a small image classification dataset.
- Implement scaled dot-product attention from scratch and verify it against a reference implementation.

Recommended tools

PyTorch as the primary deep learning framework; Hugging Face Transformers for pretrained architectures; Weights & Biases or TensorBoard for experiment tracking.

5. Module C — Generative AI Foundations

Learning objectives

- Explain how large language models are pretrained, aligned, and served.
- Distinguish between the major families of generative models and the tasks each suits.
- Reason about the tradeoffs of model size, context length, latency, and cost when selecting a model for an application.

Core topics

5.1 Foundations of large language models

- Tokenization strategies (byte-pair encoding, WordPiece, SentencePiece) and their effect on vocabulary size and multilingual coverage.
- Autoregressive next-token prediction as the pretraining objective, and how it gives rise to broad language competence.
- Scaling laws: how loss decreases predictably with model size, data, and compute, and why this motivated ever-larger models.

5.2 Alignment and instruction-tuning

- Supervised fine-tuning on instruction-response pairs.
- Reinforcement learning from human feedback (RLHF) and reward modeling.
- Direct preference optimization and other more recent alignment techniques.
- Why alignment changes a raw pretrained model's behavior from text continuation to helpful, instruction-following dialogue.

5.3 Other generative model families

- Diffusion models for image and audio generation, and the denoising process they rely on.
- Variational autoencoders and generative adversarial networks as earlier generative approaches, and why diffusion and transformer-based models have largely superseded them for most production use cases.
- Multimodal models that jointly process text, images, and audio.

5.4 Serving and inference

- Context windows and their practical limits on how much information a model can consider at once.
- Sampling strategies: temperature, top-k, and top-p (nucleus) sampling, and their effect on output diversity.
- Quantization and distillation as techniques for reducing inference cost.
- Open-weight versus hosted API models, and the operational tradeoffs of each.

Hands-on exercises

- Compare outputs from the same prompt across different temperature and top-p settings, and document how the outputs change.

- Call a hosted LLM API and a locally-served open-weight model on the same task, and compare latency, cost, and output quality.

6. Module D — Prompt Engineering and LLM Application Design

Learning objectives

- Design prompts that reliably elicit the desired structure, tone, and reasoning depth from a language model.
- Apply structured prompting techniques to reduce ambiguity and hallucination.
- Design system-level prompts for applications rather than one-off queries.

Core topics

- Zero-shot, few-shot, and chain-of-thought prompting, and when each is appropriate.
- Role and system prompt design for consistent application behavior.
- Structured output techniques: requesting JSON, XML tags, or other machine-parseable formats, and validating the result.
- Function calling and tool-use prompting, where the model chooses among a defined set of callable tools.
- Prompt templating and variable substitution for reusable, parameterized prompts.
- Common failure modes: prompt injection, ambiguous instructions, and context dilution in long prompts.

Hands-on exercises

- Write a prompt that reliably returns valid JSON matching a defined schema across twenty varied inputs.
- Compare zero-shot versus chain-of-thought prompting on a multi-step reasoning task and measure the accuracy difference.
- Design a system prompt for a customer-support assistant, including tone constraints and escalation instructions.

7. Module E — Retrieval-Augmented Generation (RAG)

Learning objectives

- Explain why retrieval-augmented generation reduces hallucination and enables models to answer questions about proprietary or fast-changing data.
- Build a complete RAG pipeline: ingestion, chunking, embedding, indexing, retrieval, and generation.
- Diagnose and address common RAG failure modes such as poor chunking or retrieval mismatch.

Core topics

7.1 Why retrieval augmentation

- The limits of parametric knowledge: models cannot know about data created after training or data never seen during training, such as private documents.
- How grounding a model's answer in retrieved passages reduces hallucination and enables citation of sources.

7.2 Document ingestion and chunking

- Parsing heterogeneous sources: PDFs, HTML, structured documents, and databases.
- Fixed-size, recursive, and semantic chunking strategies, and the tradeoff between chunk size and retrieval precision.
- Metadata attachment (source, section, date) to support filtering at query time.

7.3 Embeddings and vector stores

- Dense embedding models and how semantic similarity is measured via cosine similarity or dot product.
- Vector database options (FAISS, Chroma, Pinecone, Weaviate, pgvector) and the tradeoffs among them: latency, scalability, hybrid search support, and operational overhead.
- Approximate nearest neighbor search algorithms (HNSW, IVF) and why they trade a small amount of recall for large gains in query speed.

7.4 Retrieval strategies

- Dense retrieval, sparse (keyword-based, such as BM25) retrieval, and hybrid approaches that combine both.
- Re-ranking retrieved passages with a cross-encoder to improve precision before generation.
- Query transformation techniques: query expansion, hypothetical document embeddings (HyDE), and multi-query retrieval.

7.5 Generation and grounding

- Prompt construction that clearly separates retrieved context from the user's question and instructs the model to answer only from that context.
- Citation strategies that let the final answer reference the specific retrieved passage it relies on.
- Handling the case where retrieval returns no relevant passage, so the model declines rather than fabricates an answer.

7.6 Advanced RAG patterns

- Multi-hop RAG, where the system retrieves iteratively across several reasoning steps.
- Agentic RAG, where a controller decides whether, when, and what to retrieve, rather than retrieving on every query.

- Graph-based RAG, which retrieves from a knowledge graph in addition to or instead of unstructured text, useful for questions that require relational reasoning across entities.

Hands-on exercises

- Build an end-to-end RAG pipeline over a small document collection, including chunking, embedding, indexing, retrieval, and generation.
- Add a re-ranking step and measure the change in answer accuracy on a held-out set of questions.
- Deliberately test the pipeline with an out-of-scope question and confirm it declines rather than fabricates an answer.

8. Module F — LangChain Framework

Learning objectives

- Understand LangChain's core abstractions and how they compose into applications.
- Build chains that combine prompts, models, retrievers, and output parsers.
- Integrate external tools and memory into a LangChain application.

Core topics

8.1 Core abstractions

- Chat models and the standardized message interface across providers.
- Prompt templates and partial variables for reusable prompt construction.
- Output parsers, including structured output parsing into typed objects.
- The LangChain Expression Language (LCEL) for composing components with a consistent piping syntax, and how it supports streaming, batching, and asynchronous execution.

8.2 Retrievers and document loaders

- Document loader integrations for common file types and data sources.
- Text splitters and their relationship to the chunking strategies covered in Module E.
- Retriever interfaces that wrap vector stores, keyword search, and hybrid retrieval behind a common API.

8.3 Memory and state

- Conversation buffer and summary memory for maintaining multi-turn context.
- The distinction between short-term conversational memory and long-term persisted memory, and when each is appropriate.

8.4 Tools and agents

- Defining tools with typed inputs and clear descriptions so a model can select them correctly.

- Tool-calling agents that let a model decide which tool to invoke and with what arguments.
- Why LangChain's own agent executor has increasingly been superseded by LangGraph for applications that need explicit control flow, covered in the next module.

8.5 Integrations

- Provider-agnostic model integration, allowing an application to switch between hosted and open-weight models with minimal code changes.
- Vector store, embedding, and document loader integrations that connect the RAG concepts from Module E to working code.

Hands-on exercises

- Build a chain using LCEL that retrieves context, formats a prompt, calls a model, and parses the output into a typed object.
- Define two custom tools and build a tool-calling agent that selects between them based on the user's request.
- Add conversation memory to an application and verify that it correctly references earlier turns.

9. Module G — LangGraph: Graph-Based Orchestration

Learning objectives

- Explain why graph-based orchestration provides more reliable control flow than a purely prompt-driven agent loop.
- Model an application as a graph of nodes and edges with explicit state.
- Implement cycles, conditional branching, and human-in-the-loop checkpoints in a graph.

Core topics

9.1 Why graphs for agentic applications

- The limitation of a single linear chain: it cannot easily represent loops, branching decisions, or retries.
- How representing an application as a directed graph of nodes makes control flow explicit, inspectable, and testable, rather than implicit in a model's free-form reasoning.

9.2 Core building blocks

- State: a typed object that is passed between nodes and updated as the graph executes.
- Nodes: functions (which may call a model, a tool, or plain code) that read and update state.
- Edges: connections between nodes, including conditional edges whose destination depends on the current state.
- The graph compiler, which validates the graph structure and produces an executable application.

9.3 Cycles and control flow

- Building a cycle so a node can be revisited, for example to retry a failed tool call or to iterate until a quality check passes.
- Conditional routing based on the output of a node, such as routing to a clarification node when required information is missing.
- Parallel branches that fan out to multiple nodes and fan back in before continuing.

9.4 Persistence and human-in-the-loop

- Checkpointing graph state so a long-running or multi-day workflow can pause and resume.
- Interrupt points that pause execution for human review or approval before a sensitive action, such as sending an email or executing a financial transaction.
- Replaying and inspecting past executions for debugging, which connects directly to the tracing capabilities covered in Module I.

9.5 Multi-agent graphs

- Supervisor patterns, where one node routes work to specialized sub-agent nodes and aggregates their results.
- Hierarchical graphs, where a subgraph is itself treated as a single node in a larger graph, enabling modular composition of complex systems.

Hands-on exercises

- Model a customer-support workflow as a graph with nodes for triage, retrieval, drafting a response, and a quality-check loop that routes back to drafting if the check fails.
- Add a human-in-the-loop interrupt before any node that would take an irreversible action, and verify the graph pauses correctly.
- Build a supervisor graph that routes a request to one of three specialized sub-agents based on the request's intent.

10. Module H — Agentic AI Systems

Learning objectives

- Define what distinguishes an agentic system from a single-turn LLM call or a fixed pipeline.
- Apply established agentic design patterns to a concrete problem.
- Reason about the reliability, cost, and safety tradeoffs introduced by autonomy.

Core topics

10.1 What makes a system agentic

- The defining characteristics of an agent: it perceives a state, reasons about a goal, chooses among actions (often via tools), and observes the result before deciding on the next action.

- The spectrum from a single tool-calling step to a fully autonomous, long-running agent that plans and executes many steps with minimal supervision.

10.2 Planning and reasoning patterns

- The ReAct pattern, which interleaves reasoning traces with actions and observations.
- Plan-and-execute patterns, where a plan is drafted up front and then carried out step by step, with the option to revise the plan if a step fails.
- Reflection and self-critique patterns, where an agent evaluates its own output and revises it before returning a final answer.

10.3 Tool use and environments

- Designing tools with narrow, well-documented scopes so an agent can select them reliably.
- Sandboxing code-execution tools and other actions with side effects to limit the blast radius of a mistake.
- The Model Context Protocol (MCP) and similar standards for exposing tools and data sources to agents in a provider-agnostic way.

10.4 Multi-agent systems

- Coordination patterns: supervisor/worker hierarchies, peer-to-peer negotiation, and debate-style setups where multiple agents critique one another's output.
- The tradeoff between decomposing a task across specialized agents, which can improve quality, against the added latency and cost of multiple model calls.

10.5 Reliability, safety, and guardrails

- Bounding agent autonomy with step limits, timeouts, and cost budgets to prevent runaway loops.
- Input and output guardrails that screen for unsafe requests or unsafe generated content before an action is taken.
- Human approval gates for high-stakes or irreversible actions, echoing the human-in-the-loop pattern introduced in Module G.
- Logging and audit trails so that every action an agent takes can be reconstructed after the fact.

Hands-on exercises

- Implement a ReAct-style agent from first principles, without a framework, to understand the reasoning-action-observation loop before relying on a library's abstraction.
- Add a step limit and a cost budget to an agent and verify it halts gracefully when either is exceeded.
- Design and implement a self-critique step where the agent reviews its own draft answer against a checklist before returning it.

11. Module I — LangSmith: Observability, Tracing, and Evaluation

---

Learning objectives

- Explain why observability is essential for applications whose behavior depends on a language model's non-deterministic output.
- Instrument a LangChain or LangGraph application so every step is traced.
- Build evaluation datasets and automated evaluators to test application quality over time.

Core topics

11.1 Why observability matters for LLM applications

- The debugging challenge posed by non-deterministic, natural-language intermediate outputs, which cannot be inspected the same way as traditional program state.
- The difference between logging a final output and tracing the full sequence of prompts, tool calls, and intermediate results that produced it.

11.2 Tracing

- Automatic tracing of chains, graphs, and agent runs, capturing every prompt, model response, tool call, and latency measurement.
- Nested trace views that show how a multi-step agent or graph execution decomposes into individual runs.
- Tagging and metadata on traces to support filtering by user, environment, or feature flag.

11.3 Datasets and evaluation

- Building evaluation datasets from production traces or hand-curated examples.
- Running an application against a fixed dataset to produce reproducible comparisons across prompt or model changes.
- LLM-as-judge evaluators, where a separate model call scores an output against a rubric, and the calibration work needed to trust such a judge.
- Heuristic and rule-based evaluators for checks that do not require a model call, such as format validation or the presence of required fields.

11.4 Feedback and monitoring in production

- Capturing user feedback (thumbs up/down, corrections) and attaching it to the corresponding trace.
- Dashboards for tracking latency, cost, and error rate over time, and alerting on regressions.
- Using production traces to continuously grow the evaluation dataset, closing the loop between monitoring and evaluation.

11.5 Prompt and experiment management

- Versioning prompts so changes can be tracked, compared, and rolled back.
- Running side-by-side experiments across prompt or model variants against the same evaluation dataset.

Hands-on exercises

- Instrument an existing LangGraph application so that every node's input and output is traced.
- Build a twenty-example evaluation dataset for a RAG application and run it through both a heuristic evaluator and an LLM-as-judge evaluator.
- Change a single prompt in an application and use tracing to compare the before-and-after behavior on the same evaluation dataset.

12. Module J — Model Evaluation Methodology

Learning objectives

- Select evaluation metrics appropriate to a task, whether classical, deep learning, or generative.
- Design an evaluation protocol that avoids common pitfalls such as data leakage and benchmark overfitting.
- Evaluate generative and agentic systems along dimensions beyond simple accuracy, including safety, cost, and latency.

Core topics

12.1 Evaluating classical and deep learning models

- Revisiting classification and regression metrics from Module A in the context of deep models, including calibration and confidence estimation.
- Held-out test sets, cross-validation, and the danger of tuning decisions on the test set.
- Subgroup evaluation to detect performance disparities across demographic or data segments.

12.2 Evaluating generative text output

- Reference-based metrics such as BLEU and ROUGE, and their well-documented limitations for open-ended generation.
- Embedding-based similarity metrics as a softer alternative to exact n-gram overlap.
- Human evaluation protocols, including pairwise preference judgments and Likert-scale rubrics.
- LLM-as-judge evaluation, its correlation with human judgment, its known biases (such as favoring longer or more confidently worded answers), and mitigations such as randomizing answer order and using multiple judge models.

12.3 Evaluating retrieval and RAG systems

- Retrieval metrics: precision at k, recall at k, and mean reciprocal rank.
- End-to-end RAG metrics: faithfulness (whether the answer is supported by retrieved context), answer relevance, and context precision.
- Distinguishing retrieval failures from generation failures during error analysis, since the fix for each is different.

12.4 Evaluating agentic systems

- Task success rate on held-out tasks with known correct outcomes.
- Trajectory evaluation: whether the sequence of tool calls and intermediate reasoning steps was efficient and sensible, not just whether the final answer was correct.
- Cost and latency per task, since an agent that succeeds but takes fifty steps may be unusable in production.
- Safety evaluation: red-teaming an agent with adversarial inputs to check whether guardrails hold under pressure.

12.5 Benchmarking practices

- Public benchmark suites and their appropriate use as a rough signal rather than a complete substitute for task-specific evaluation.
- The risk of benchmark contamination, where evaluation data has leaked into a model's training set, and how to check for it.
- Building a private, task-specific evaluation set that reflects real usage, which typically matters more than public leaderboard position for a production application.

Hands-on exercises

- Design a rubric for LLM-as-judge evaluation of a summarization task, and validate it against a small set of human-labeled examples.
- Compute precision at k and recall at k for a retrieval system, then correlate those numbers with end-to-end answer quality.
- Run a trajectory evaluation on an agent's transcript, identifying any unnecessary or redundant tool calls.

13. Module K — Deployment, MLOps, and LLMOps

Learning objectives

- Package and deploy a trained model or LLM application behind a stable API.
- Apply operational practices that keep a model-driven system reliable, observable, and cost-controlled after launch.

Core topics

- Model packaging and serving: containerization, model registries, and serving frameworks for classical and deep learning models.
- API design for LLM-backed applications, including streaming responses and handling partial failures gracefully.
- Caching strategies for repeated prompts or retrieval queries to reduce cost and latency.
- Rate limiting, retry logic, and fallback models for handling upstream provider outages or throttling.

- Continuous evaluation in CI/CD: running the evaluation datasets from Module I and Module J automatically before every deployment.
- Cost monitoring and budget alerts, since token-based pricing can scale unpredictably with usage.
- Data privacy and compliance considerations when passing user data to third-party model providers.

Hands-on exercises

- Containerize a RAG application and deploy it behind a simple API with streaming responses.
- Add a caching layer for repeated queries and measure the resulting latency and cost reduction.
- Wire the evaluation dataset from Module I into a CI pipeline so that a pull request fails automatically if evaluation scores regress beyond a defined threshold.

14. Capstone Projects

Capstone projects are designed to integrate every module into a single working system. Learners should choose at least one project and carry it from design through evaluation.

Project 1: Document Question-Answering Assistant

- Build a RAG pipeline over a real document collection (for example, internal documentation or a public regulatory corpus).
- Implement hybrid retrieval with re-ranking, and add citations to every answer.
- Instrument the pipeline with tracing and build an evaluation dataset covering faithfulness and answer relevance.

Project 2: Multi-Agent Research Assistant

- Design a LangGraph application with a supervisor node that routes sub-tasks to a web-research agent, a summarization agent, and a fact-checking agent.
- Add a human-in-the-loop approval step before the final report is published.
- Evaluate the system on task success rate and trajectory efficiency.

Project 3: Customer Support Automation

- Build an agent that classifies incoming support tickets, retrieves relevant knowledge-base articles, and drafts a response for human review.
- Add guardrails that route sensitive requests (refunds above a threshold, legal questions) directly to a human agent.
- Set up continuous evaluation using production feedback captured through tracing.

Project 4: Classical-to-Deep Learning Benchmark Study

- Select a single tabular or text classification task and compare a classical model, a fine-tuned deep learning model, and a prompted LLM on identical evaluation splits.

- Report accuracy, latency, and cost for each approach, and make a recommendation for which is best suited to production given the tradeoffs observed.

15. Suggested 16-Week Study Schedule

Weeks 1–2: Prerequisites review; Module A — Machine Learning Fundamentals.

Weeks 3–4: Module B — Deep Learning, including the transformer architecture.

Week 5: Module C — Generative AI Foundations.

Week 6: Module D — Prompt Engineering and LLM Application Design.

Weeks 7–8: Module E — Retrieval-Augmented Generation, including the capstone RAG exercise.

Weeks 9–10: Module F — LangChain Framework.

Weeks 11–12: Module G — LangGraph, including cycles, human-in-the-loop, and multi-agent graphs.

Week 13: Module H — Agentic AI Systems.

Week 14: Module I — LangSmith: tracing, datasets, and evaluators.

Week 15: Module J — Model Evaluation Methodology, and Module K — Deployment, MLOps, and LLMOps.

Week 16: Capstone project completion and final review.

This schedule assumes roughly eight to ten hours of study per week, including reading, hands-on exercises, and capstone work. Learners with less time available should extend the schedule rather than skip the hands-on exercises, since the exercises are where the material is actually internalized.

16. Assessment Rubric and Certification Checklist

Use the checklist below to confirm readiness before considering the curriculum complete. Each item should be demonstrable with working code or a written artifact, not just conceptual familiarity.

- Can train, evaluate, and explain the tradeoffs of at least two classical machine learning models on a real dataset.
- Can implement backpropagation for a small neural network without relying on a deep learning framework's autograd.
- Can explain self-attention and implement scaled dot-product attention from scratch.
- Can articulate how a large language model is pretrained, aligned, and served, including the role of RLHF or similar techniques.
- Can design prompts that reliably produce structured, schema-conformant output.
- Can build a complete RAG pipeline, including chunking, embedding, retrieval, re-ranking, and grounded generation with citations.
- Can compose a LangChain application using LCEL, including custom tools and memory.
- Can design a LangGraph application with conditional edges, a cycle, and a human-in-the-loop interrupt.

- Can implement an agent using a recognized planning pattern (ReAct or plan-and-execute) with explicit step and cost limits.
- Can instrument an application with LangSmith tracing and build an evaluation dataset with both heuristic and LLM-as-judge evaluators.
- Can select and justify appropriate evaluation metrics for a classical model, a generative model, a RAG system, and an agentic system.
- Can deploy an application behind an API with caching, retries, and continuous evaluation wired into a deployment pipeline.
- Has completed at least one capstone project end to end, from design through evaluation.

17. Reference Library and Further Reading

The following categories of material are useful for deepening study beyond this curriculum. Rather than naming specific editions or URLs that can go out of date, each category below describes the kind of source to seek out and why it is useful.

Foundational textbooks and courses

- A standard machine learning textbook covering statistical learning theory, for the mathematical grounding behind Module A.
- A deep learning textbook covering optimization, convolutional networks, sequence models, and the transformer architecture, for Module B.
- University-level courses on natural language processing and large language models for deeper treatment of Module C material.

Primary research papers

- The original transformer paper for the attention mechanism covered in Module B.
- Papers introducing instruction-tuning and reinforcement learning from human feedback for the alignment concepts in Module C.
- Papers introducing the ReAct prompting pattern and subsequent agentic planning approaches for Module H.
- Papers proposing RAG evaluation metrics such as faithfulness and answer relevance for Module J.

Official framework documentation

- LangChain's official documentation for up-to-date API references, integration guides, and conceptual guides referenced throughout Module F.
- LangGraph's official documentation for graph construction, persistence, and multi-agent patterns referenced in Module G.
- LangSmith's official documentation for tracing, dataset, and evaluator APIs referenced in Module I.
- Vector database and embedding provider documentation for the specific tools chosen during the Module E exercises.

Communities and continued learning

- Open-source repositories of example agentic applications, useful for seeing production-grade patterns beyond the exercises in this document.
- Engineering blogs from organizations operating large-scale LLM applications, which often describe evaluation and reliability practices in more operational detail than academic papers.
- Public leaderboards and benchmark trackers, used cautiously and in combination with the task-specific evaluation practices covered in Module J rather than as a sole source of truth.

End of curriculum document.